

Drug-Target Binding Affinity Prediction

1. Executive Summary & What We're Actually Trying to Solve

1.1 The Problem, in Plain Terms

Structure-based drug discovery lives and dies on one question: how tightly does this ligand bind that protein? That “tightness” usually shows up as pKd, the negative log of the dissociation constant. If you can predict pKd well before touching a wet lab, you can rank candidates, ditch the weak ones early, and spend real money only where it counts. That's the whole point of virtual screening and CADD.

Formally it's simple: given a pocket P and ligand L, learn a function $f(P, L) \rightarrow \hat{y}$, where $\hat{y}$ tracks the experimental pKd.

But the real interaction isn't “simple descriptors in, number out.” Binding depends on 3D complementarity, electrostatics, and lots of picky non-covalent contacts between particular atoms and residues. If you only feed ligand fingerprints or a SMILES string, you're basically asking the model to guess the pocket. That's not enough — you need a joint structural view of both sides.

1.2 What Makes This Annoying in Practice

1.2.1 Data Quality Is Messier Than People Admit

PDBBind is useful. It's also inconsistent — different file quirks, odd provenance, broken ligands, half-empty pockets, you name it. So the pipeline starts with a hard, auditable verification gate. No silent failures.

A complex only passes if:

- the pocket structure parses cleanly and contains at least one residue, and
- the ligand file (MOL2 preferred, SDF as backup) produces a valid RDKit molecule with at least one atom.

Fail either check? It's out. Logged, with a machine-readable reason. That way graph building only touches complexes that are actually sane, and every exclusion is traceable without playing detective later.

1.2.2 Generalization Bias (aka the “Scaffold Leakage” Trap)

Random splitting over protein–ligand complexes looks fine on paper. In binding affinity datasets, it's a classic mistake. Why? Because PDBBind has tons of congeneric series: same core scaffold, tiny substituent tweaks. A naive random split happily puts close cousins into both train and validation. Then the model “generalizes” by recognizing familiar scaffolds instead of learning transferable structure–activity rules. Validation error drops. You feel good. It's kind of a lie.

So the pipeline splits by Bemis–Murcko scaffold. Every complex sharing a scaffold goes into the same partition. Train never sees scaffolds that validation later uses. That's not a nice-to-have — it's the minimum bar for a credible out-of-distribution estimate. And CASF-2013? That stays completely isolated. No training. No tuning. Not even “just a quick peek.”

2. Proposed System Workflow (Phases 1–3)

This is built as a sequential, checkpointed pipeline. Each phase writes artifacts to disk — verification ledgers, serialized graph tensors, split manifests — so you can rerun later stages without redoing everything upstream.

Phase 1 — Ingestion & Verification

Each complex (by PDB ID) is expected to provide a protein file, a pocket file, and a ligand file (MOL2, with SDF fallback). Validation uses two libraries, each doing what it's good at:

- BioPython (Bio.PDB.PDBParser) parses the pocket PDB and checks there's at least one residue across models/chains. Empty pockets get rejected. So do malformed structures.
- RDKit parses and sanitizes the ligand, trying MOL2 first (Chem.MolFromMol2File) and falling back to SDF (Chem.SDMolSupplier). Sanitization failures or zero-atom molecules? Rejected.

Every entry gets a valid flag, an error message if invalid, and a source label (refined vs. core). The full ledger is exported as `verified_complexes.csv`. No mystery meat.

Phase 2 — Graph Construction

2.1 Ligand Graph

Each verified ligand becomes a molecular graph $G_L = (V_L, E_L)$. Each atom gets a 5-d feature vector: atomic number, degree, formal charge, aromatic flag, hybridization state.

```
x_atom = [atomic_number, degree, formal_charge, is_aromatic, hybridization]
```

Per-atom feature vector, extracted via RDKit's Atom API.

Bonds become bidirectional edges. For bond (i, j) , add $(i \rightarrow j)$ and $(j \rightarrow i)$. Each directed edge carries one scalar attribute: the RDKit bond order (1.0, 1.5 aromatic, 2.0, 3.0). Message passing flows both ways without needing the layer itself to “guess” directionality.

2.2 Pocket Graph (Residue-Level)

The pocket is modeled as a residue contact graph, not an all-atom graph. That keeps things smaller while still encoding the binding context.

- Node features: a 20-d one-hot over standard amino acids (ALA...VAL). Non-canonical residues map to an all-zero vector.
- Coordinates: each residue uses its Ca position.
- Edges: residues i and j connect if their Ca distance is within 8Å.

```
edge(i, j)  $\Leftarrow$  || CA_i - CA_j ||2  $\leq$  8.0 Å
```

Yes, this is an $O(n^2)$ all-pairs check in the basic version — also a very common structural bioinformatics convention for “close enough to matter” contacts without going full atomic.

Phase 3 — Scaffold-Based Splitting

For each refined-set complex: compute the ligand's Bemis–Murcko scaffold via RDKit (`MurckoScaffold.GetScaffoldForMol`), then canonicalize the scaffold to a SMILES string. If scaffold extraction fails, the complex becomes its own singleton scaffold group — it still stays in the dataset, it just won't accidentally merge with anything else.

Then scaffold groups (not individual complexes) get greedily assigned: biggest scaffold groups first, fill training until ~80% of complexes are covered, and the rest goes to validation. A set-intersection sanity check verifies there is zero scaffold overlap between train and validation.

CASF-2013 (193 complexes) bypasses this entire process. It remains a locked benchmark partition end-to-end.

3. Model Architecture (Phase 4): Dual GNN + Bidirectional Cross-Attention

The model (`BindingAffinityModel`) is implemented in PyTorch + PyTorch Geometric. Two encoders run in parallel — one for the ligand graph, one for the pocket graph. Then a cross-attention fusion block lets them “talk” at node level. Finally, pooled embeddings go through an MLP to regress a single pKd value.

```
HIDDEN_DIM = 128
NUM_MOL_LAYERS = 3      # GIN
NUM_POCKET_LAYERS = 3   # GAT
ATTENTION_HEADS = 4
DROPOUT = 0.2
```

Key hyperparameters.

3.1 Ligand Encoder: GIN Stack

The ligand branch uses Graph Isomorphism Network (GIN) layers. Reason: GIN's sum aggregation + MLP is about as discriminative as message-passing GNNs get under the Weisfeiler–Lehman lens, which matters when you're trying to separate “almost the same” small molecules (regioisomers, subtle topology changes, etc.). Mean/max aggregators can blur those.

Workflow: project raw 5-d atom features to 128-d, then apply 3 GINConv layers, each wrapping a 2-layer MLP ($128 \rightarrow 128 \rightarrow 128$). After each layer: ReLU, dropout (0.2), residual skip.

```
 $x \in \mathbb{R}^{N \times 5} \rightarrow \text{Linear}(5, 128) \rightarrow \text{ReLU} \rightarrow h_0 \in \mathbb{R}^{N \times 128}$ 
 $h_{l+1} = h_l + \text{Dropout}(\text{ReLU}(\text{GINConv}_l(h_l, E_L)))$ 
```

That residual path isn't decoration — it keeps gradients stable and preserves lower-level atom identity while deeper layers inject neighborhood context.

3.2 Pocket Encoder: GAT Stack

Pocket graphs are different beasts. They're denser. Edges come from spatial proximity, not chemistry bonds. Treating every neighbor equally (sum) is a blunt instrument. So the pocket branch uses Graph Attention Networks (GAT) — attention lets the model learn which neighboring residues are actually worth listening to.

Steps: project 20-d residue one-hot to 128-d, then run 3 GATConv layers (heads=4, concat=False so heads average and output stays 128-d, dropout=0.2). Each layer wrapped with ReLU + dropout + residual, mirroring the ligand branch so both streams reach fusion with comparable 128-d node embeddings.

```
GATConv(128, 128, heads=4, concat=False, dropout=0.2)
```

3.3 Cross-Attention Fusion (the Main Trick)

Here's where the two graphs stop being strangers. A bidirectional cross-attention module (CrossAttentionFusion) uses two nn.MultiheadAttention blocks (4 heads, batch_first=True): mol→pocket attention (ligand nodes query pocket nodes) and pocket→mol attention (pocket nodes query ligand nodes).

```
MolAttended    = MHA_mp( Q=Mol,      K=Pocket, V=Pocket )
PocketAttended = MHA_pm( Q=Pocket, K=Mol,   V=Mol   )

Mol_fused      = Mol      + MolAttended
Pocket_fused   = Pocket + PocketAttended
```

What does that buy you? A learned, differentiable way to highlight which atoms and residues matter for this specific complex, without hand-coding an atom-residue contact map at inference time. The attention weights come from scaled dot-product similarity between Q and K, computed fresh per sample.

3.4 Pooling + Regression Head

Node embeddings are variable-length, so they need to become fixed-size vectors. This implementation uses global mean pooling separately for each branch, concatenates the two into a 256-d joint vector, and passes it through a 3-layer MLP:

```
mol_pooled      = mean(Mol_fused,      over nodes) →  $\mathbb{R}^{1 \times 128}$ 
pocket_pooled   = mean(Pocket_fused, over nodes) →  $\mathbb{R}^{1 \times 128}$ 

Linear(256 → 128) → ReLU → Dropout(0.2)
Linear(128 → 64)  → ReLU
Linear(64 → 1)    # predicted pKd
```

Final layer is linear (no activation). Output is an unconstrained real value, trained end-to-end with MSE against experimental pKd.

4. Future Upgrades (High-Impact, Not Fluff)

4.1 Put Real 3D Geometry into the Model (Equivariant GNNs)

Right now the graphs are mostly topological: ligand edges from bonds, pocket edges from an 8Å cutoff. But there's no learnable use of angles, dihedrals, or precise distances. Moving to E(3)-equivariant models like EGNN or SchNet would let the network consume coordinates directly while staying rotation/translation invariant. That's where binding physics lives.

4.2 Use Pretrained Chemical/Protein Embeddings

Current node features are hand-built and small: 5-d atoms, 20-d residues. You could inject much richer priors by pulling embeddings from ChemBERTa (ligand substructures/atoms) and ESM-2 (protein

residues). Given refined-set scale (thousands of complexes), transfer learning isn't optional if you want the ceiling to move.

4.3 Make Attention Aware of Edges / Distances

Cross-attention currently depends only on node embeddings, yet we already have ligand bond order and potential inter-node distances (if we include coordinates). Adding distance-aware or edge-conditioned bias terms to attention logits (pre-softmax) would push attention to respect physical interaction ranges instead of learning them indirectly.

4.4 Multi-Task Learning

Only regressing pKd can leave representations under-constrained. Add auxiliary heads — pocket druggability classification, ligand solubility prediction, maybe even contact prediction if labels can be approximated. Shared encoders + weighted multi-task loss can improve generalization beyond PDBBind's label quirks.

4.5 Auxiliary Toxicity Prediction for Combined Molecules

A further scalability extension is a joint toxicity-prediction head that scores the combined ligand–pocket complex (or the ligand in the context of multi-compound / combination-therapy scenarios) alongside the primary pKd regression target. Architecturally, this reuses the existing fused representation — the concatenated mol_pooled and pocket_pooled vectors already computed by the fusion layer — and routes it through a second lightweight MLP head in parallel with the existing regression head, rather than requiring a separate encoder stack:

```
toxicity_logits = ToxicityHead( concat(mol_pooled, pocket_pooled) )  
Linear(256 → 128) → ReLU → Dropout(0.2) → Linear(128 → T) # T toxicity endpoints
```

Proposed multi-endpoint toxicity head (e.g., hepatotoxicity, cardiotoxicity/hERG, mutagenicity), trained as multi-label binary classification with a weighted BCE loss added to the existing MSE pKd loss.

For combined-molecule (drug–drug or multi-ligand) scenarios, this can be extended by encoding each additional compound through the same shared molecule encoder (weight-tied GIN), mean-pooling across the set of per-molecule embeddings before concatenation with the pocket representation — so the toxicity head naturally accepts a variable number of co-administered molecules without architectural changes. Public toxicity resources such as Tox21, ClinTox, and DrugBank drug–drug interaction labels are natural sources of auxiliary supervision for pre-training or joint fine-tuning of this head. Framed this way, toxicity prediction becomes a second instance of the multi-task learning strategy from Section 4.4, sharing the same structural encoders and amortizing their training cost across both objectives.

4.6 Engineering for Scale

A few practical upgrades that matter:

- switch from per-complex training (batch size 1) to real batched graph training using PyG DataLoader/Batch
- add LR scheduling (cosine anneal or ReduceLROnPlateau) and weight decay to stabilize bigger runs
- speed up pocket-edge building: replace $O(n^2)$ loops with KD-trees or torch.cdist neighbor search